

Ferramentas de Apoio ao Git - Aula 7

🕒 Criado em	9 de setembro de 2026 15:01
🏷️ Tags	

1. Visão geral da aula

Até aqui, o Git foi apresentado principalmente como uma ferramenta para:

- controlar versões;
- registrar alterações;
- criar branches;
- fazer commits;
- integrar mudanças;
- trabalhar em equipe.

Nesta aula, a ideia é dar um passo além.

O Git permite criar mecanismos que executam determinadas verificações automaticamente durante o fluxo de desenvolvimento.

Esses mecanismos são chamados de:

hooks do Git.

A partir deles, podemos utilizar ferramentas como:

- pre-commit;
- Commitizen;
- detect-secrets;
- formatadores;
- linters;
- verificadores de arquivos;
- verificadores de conflitos.

A ideia geral é:

```
Desenvolvedor altera código
↓
Git detecta determinada ação
↓
Hook é executado
↓
Validações são realizadas
↓
Se estiver tudo correto
↓
Git continua a operação
```

Isso permite transformar regras do projeto em verificações automáticas.

2. O problema que essas ferramentas tentam resolver

Imagine uma equipe com 10 desenvolvedores.

O projeto possui algumas regras:

- não colocar senhas no código;
- não adicionar arquivos gigantes;
- não deixar espaços desnecessários;
- seguir um padrão de commit;
- evitar conflitos;
- manter o código formatado.

Uma possibilidade seria escrever no documento:

|"Pessoal, lembrem-se de seguir todas essas regras."

O problema é que seres humanos esquecem coisas.

Além disso, novos desenvolvedores podem nem conhecer todas as regras.

Então surge uma ideia melhor:

Em vez de depender apenas da memória e disciplina das pessoas, podemos automatizar algumas verificações.

É exatamente aqui que entram os hooks e ferramentas de apoio.

3. O que é um Git Hook?

Um **hook** é um mecanismo que permite executar alguma ação automaticamente quando determinado evento acontece no Git.

A palavra "hook" pode ser entendida como:

| um "gancho" conectado a uma etapa do fluxo.

Por exemplo:

```
git commit
  ↓
hook
  ↓
verificação
  ↓
commit
```

Ou:

```
commit-msg
  ↓
verifica mensagem
  ↓
aceita ou rejeita
```

Assim, podemos associar verificações a determinadas etapas do Git.

4. Por que usar hooks?

Porque determinadas regras são tão importantes que não queremos depender apenas da lembrança do desenvolvedor.

Imagine:

```
Regra:
"Não coloque uma chave de API no código."
```

Podemos transformar isso em:

```
Código
↓
pre-commit
↓
detect-secrets
↓
encontrou segredo?
↓
SIM → bloqueia commit
```

Agora a regra deixa de ser apenas uma recomendação.

Ela passa a ser uma verificação automática.

5. Configuração inicial

A aula começa criando um repositório:

```
git init demo-repo
```

Esse comando cria um novo repositório Git.

Depois são configurados:

```
git config --global user.email "you@example.com"
git config --global user.name "Your Name"
```

Essas configurações identificam o autor dos commits.

É importante lembrar:

```
user.name
user.email
```

estão relacionados à identidade registrada nos commits.

Isso é diferente da autenticação utilizada para acessar um repositório remoto.

6. Instalação das ferramentas

A aula instala:

```
pip install pre-commit commitizen detect-secrets
```

Temos três ferramentas principais:

pre-commit

Gerencia e executa hooks.

Commitizen

Ajuda a padronizar mensagens de commit.

detect-secrets

Ajuda a identificar possíveis segredos no código.

Elas podem trabalhar juntas.

7. Pre-commit

O `pre-commit` é um framework para gerenciamento de hooks.

A ideia principal é:

Configurar verificações que serão executadas automaticamente antes de determinadas operações do Git.

Um arquivo muito importante é:

```
.pre-commit-config.yaml
```

Esse arquivo descreve quais verificações o projeto deseja utilizar.

8. Por que existe um arquivo de configuração?

Imagine que uma equipe possui:

Ana
João

```
Maria
Carlos
```

Se cada pessoa configurasse suas ferramentas manualmente, poderiam surgir diferenças:

```
Ana → regra A
João → regra B
Maria → regra A + C
Carlos → nenhuma regra
```

Isso seria um problema.

O arquivo:

```
.pre-commit-config.yaml
```

permite que o projeto declare suas regras.

Assim, a equipe consegue compartilhar a mesma configuração.

A ideia é:

```
Regras do projeto
  ↓
Arquivo de configuração
  ↓
Todos os desenvolvedores
  ↓
Mesmo conjunto de verificações
```

9. Exemplo de configuração

O material apresenta:

```
repos:
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v5.0.0
  hooks:
    - id: trailing-whitespace
    - id: end-of-file-fixer
```

```
- repo: https://github.com/Yelp/detect-secrets
  rev: v1.5.0
  hooks:
    - id: detect-secrets
```

Não é necessário decorar toda a sintaxe.

O importante é entender a estrutura conceitual.

Temos:

```
repos
  ↓
repositórios que fornecem hooks
  ↓
hooks
  ↓
verificações que queremos executar
```

10. O hook trailing-whitespace

O:

```
trailing-whitespace
```

verifica espaços em branco desnecessários no final das linhas.

Por exemplo, imagine:

```
nome = "Ana" ____
```

Os caracteres  representam espaços que ficaram depois do código.

Esses espaços geralmente não possuem valor para o programa.

Podem ser removidos automaticamente.

A ferramenta ajuda a manter o código mais limpo.

11. O hook end-of-file-fixer

Esse hook verifica problemas relacionados ao final dos arquivos.

A ideia é padronizar o final do arquivo para evitar inconsistências.

Esse tipo de regra parece pequeno.

Mas imagine um projeto com centenas de arquivos e vários desenvolvedores.

Se cada pessoa configurar os arquivos de uma maneira diferente, o repositório pode acumular pequenas inconsistências.

Automatizar essas regras reduz esse tipo de problema.

12. Detect-secrets

Esse é um dos exemplos mais importantes da aula.

O objetivo é detectar possíveis informações secretas dentro do código.

Por exemplo:

```
API_KEY = "alguma-chave-secreta"
```

Ou:

```
TOKEN = "..."
```

Ou credenciais que não deveriam estar no código-fonte.

Isso é muito perigoso porque o código pode ser enviado para um repositório remoto.

Se o repositório for compartilhado ou público, a informação pode acabar exposta.

13. O fluxo do exemplo de segredo

O material cria um arquivo:

```
config.py
```


com uma possível API key.

Depois executa:

```
git add config.py
git commit -m "Commit inicial"
```

O que acontece?

Antes do commit ser concluído, o hook é executado.

O `detect-secrets` analisa o arquivo.

Se identificar algo suspeito, o processo pode ser interrompido.

Podemos visualizar:

```
git commit
  ↓
pre-commit
  ↓
detect-secrets
  ↓
segredo encontrado
  ↓
commit bloqueado
```

Isso é uma barreira de segurança.

14. O que acontece depois que corrigimos o problema?

O material remove a possível chave:

```
KEY = None
```

Depois tenta novamente.

Agora o problema foi removido.

O hook pode concluir a verificação.

Então:

```
git commit
  ↓
verificações
  ↓
sem problema
  ↓
commit permitido
```

Essa é uma das grandes vantagens dos hooks:

Eles podem impedir que um problema conhecido avance para a próxima etapa.

15. Uma ideia importante: prevenção

Observe a diferença:

Sem automação:

```
Desenvolvedor
  ↓
commit
  ↓
GitHub
  ↓
problema descoberto
```

Com hook:

```
Desenvolvedor
  ↓
commit
  ↓
verificação
  ↓
problema encontrado
  ↓
corrigir
```

O problema é descoberto mais cedo.

Quanto mais cedo um problema for encontrado, normalmente menor será o custo para corrigi-lo.

Isso se conecta a um princípio muito importante da Engenharia de Software:

Detectar problemas cedo tende a ser melhor do que descobrir o problema depois que ele já avançou no processo.

16. Mas existe uma limitação muito importante

O `pre-commit` local possui uma limitação:

Ele depende do ambiente local do desenvolvedor.

Imagine:

```
Ana → instalou pre-commit  
João → instalou pre-commit  
Maria → não instalou  
Carlos → desativou
```

Ana e João terão verificações.

Maria e Carlos podem conseguir criar commits sem passar pelas mesmas validações.

Portanto:

```
pre-commit local  
≠  
garantia absoluta
```

Essa é uma parte muito importante da aula.

17. Por que isso acontece?

Porque o hook está instalado na máquina do desenvolvedor.

O Git executa aquele hook localmente.

Se ele não estiver instalado, não existe aquela barreira naquele ambiente.

Então temos:

```
Computador A
  ↓
hook instalado
  ↓
verificação

Computador B
  ↓
hook não instalado
  ↓
nenhuma verificação local
```

Isso cria uma diferença entre os ambientes.

18. Como resolver isso em uma equipe?

A solução não é abandonar o pre-commit.

Ele continua sendo muito útil.

Mas devemos combiná-lo com outras camadas.

Por exemplo:



Ou seja:

camada local

Ajuda o desenvolvedor a detectar o problema rapidamente.

camada de integração contínua

Confere novamente no servidor.

19. Integração contínua

Integração Contínua, ou:

CI — Continuous Integration

é uma prática em que alterações enviadas ao repositório passam automaticamente por verificações.

Por exemplo:

```
Push
↓
CI
↓
Build
↓
Testes
↓
Lint
↓
Verificação de segurança
↓
Resultado
```

Se algum requisito falhar, a alteração pode ser impedida de avançar.

20. Por que CI complementa o pre-commit?

Porque o pre-commit atua principalmente no computador do desenvolvedor.

A CI atua em um ambiente controlado pelo projeto/empresa.

Então:

```
Pre-commit
→ prevenção local

CI
→ validação coletiva
```

Mesmo que alguém não tenha executado corretamente o hook local, a CI pode detectar o problema.

Essa é uma ideia muito importante para ambientes profissionais:

Regras importantes não devem depender exclusivamente da disciplina individual.

21. Validação no servidor remoto

Além da CI, plataformas de hospedagem de código podem aplicar regras no processo de integração.

Por exemplo, uma equipe pode configurar o projeto para exigir:

```
Pull Request
  ↓
CI passou?
  ↓
Sim → pode continuar
Não → bloqueado
```

Também pode exigir:

- revisão de código;
- aprovação;
- testes;
- branches protegidas;
- verificações obrigatórias.

Assim, temos várias camadas de proteção.

22. A ideia de defesa em profundidade

Esse conceito pode ser entendido assim:

```
Camada 1
Pre-commit
  ↓
Camada 2
CI
  ↓
Camada 3
Regras do repositório
  ↓
```

Se uma camada falhar, outra pode detectar o problema.

Isso é muito mais seguro do que:

```
"Espero que todo mundo execute o pre-commit."
```

23. Commitizen

Agora entra outra ferramenta:

Commitizen.

Seu objetivo é ajudar a criar mensagens de commit padronizadas.

Isso é importante porque o histórico do Git é uma fonte de informação sobre a evolução do projeto.

Compare:

```
alterações  
coisas  
mudança  
final  
agora vai  
teste
```

com:

```
feat: adiciona autenticação  
fix: corrige validação de login  
docs: atualiza documentação
```

A segunda forma transmite muito mais informação.

24. Conventional Commits

O material utiliza o padrão:

Conventional Commits.

Alguns tipos comuns são:

```
feat:  
fix:  
docs:
```

Podemos interpretar:

feat

Nova funcionalidade.

```
feat: adiciona cadastro de usuário
```

fix

Correção de problema.

```
fix: corrige validação de senha
```

docs

Alteração na documentação.

```
docs: atualiza instruções de instalação
```

O objetivo é criar um histórico previsível e compreensível.

25. Por que padronizar commits?

Imagine um projeto com centenas ou milhares de commits.

Se todos seguirem padrões diferentes:

```
nova função  
adicionando login  
login pronto  
fix login  
corrige bug  
bug do login corrigido
```


fica difícil analisar o histórico.

Com um padrão:

```
feat: adiciona login  
fix: corrige autenticação  
docs: documenta login
```

fica mais fácil identificar a natureza das alterações.

Isso também pode facilitar automações.

Por exemplo, ferramentas podem analisar os tipos de commit para ajudar na geração de changelogs.

26. Por que usar Commitizen?

Uma equipe poderia simplesmente dizer:

| "Todo mundo deve decorar o padrão."

Mas isso cria dificuldade principalmente para iniciantes.

O Commitizen pode guiar o desenvolvedor na criação da mensagem.

A ideia é:

```
Commit  
↓  
Commitizen  
↓  
Perguntas/opções  
↓  
Mensagem padronizada
```

Assim, a ferramenta ajuda a pessoa a seguir a convenção.

27. O papel do pre-commit com Commitizen

Aqui temos uma integração interessante.

O projeto pode configurar o Commitizen como um hook de:

```
commit-msg
```

Esse tipo de hook está relacionado especificamente à mensagem do commit.

Então podemos ter:

```
git commit
  ↓
mensagem criada
  ↓
commit-msg hook
  ↓
verifica padrão
  ↓
mensagem correta?
```

Se não estiver correta:

```
commit bloqueado
```

28. Exemplo da aula

A mensagem:

```
mensagem sem padrão
```

não segue o padrão esperado.

Então o hook rejeita.

Depois:

```
feat: adicionando hello world
```

segue o padrão.

Então o commit pode continuar.

A ideia importante é:

O Git não está simplesmente armazenando qualquer mensagem; o projeto pode definir regras para a forma como seu histórico deve ser construído.

29. O significado de `stages: [commit-msg]`

Essa configuração é importante conceitualmente.

Existem diferentes momentos no processo de commit.

O projeto pode executar uma verificação:

```
antes do commit
```

ou especificamente:

```
na validação da mensagem do commit
```

Quando temos:

```
stages: [commit-msg]
```

estamos dizendo que aquele hook deve atuar na etapa relacionada à mensagem do commit.

Isso faz sentido para Commitizen porque o que ele precisa verificar é:

"A mensagem desse commit está de acordo com a convenção?"

30. O comando de instalação específico

A aula apresenta:

```
pre-commit install --hook-type commit-msg
```

Isso instala o hook associado ao momento de validação da mensagem.

É diferente de simplesmente instalar os hooks padrão.

Conceitualmente:

```
pre-commit install
```

→ instala os hooks padrão configurados.

```
pre-commit install --hook-type commit-msg
```

→ instala também o mecanismo necessário para executar verificações na mensagem do commit.

31. Caso de uso real: arquivos grandes

Agora a aula apresenta um problema bastante comum.

Imagine que alguém tenha um arquivo de:

```
500 MB
```

e execute:

```
git add arquivo.bin  
git commit -m "Adicionando arquivo"
```

O problema é que o arquivo pode acabar entrando no histórico do Git.

Isso pode:

- aumentar muito o tamanho do repositório;
- deixar clones mais lentos;
- dificultar armazenamento;
- tornar operações do Git mais pesadas.

Por isso, é interessante detectar arquivos grandes antes do commit.

32. **check-added-large-files**

O hook:

```
check-added-large-files
```

verifica arquivos grandes que estão sendo adicionados.

A aula utiliza:

```
args: [ '- --maxkb=1024' ]
```

Isso estabelece:

```
1024 KB
```

como limite.

Aproximadamente:

```
1 MB
```

Assim:

```
arquivo < 1 MB  
→ permitido  
  
arquivo > 1 MB  
→ pode ser bloqueado
```

33. Por que isso é importante?

Imagine que alguém sem querer adicione:

```
video.mp4
```

com:

800 MB

ao projeto.

Talvez a pessoa nem perceba.

Mas o Git pode passar a carregar esse arquivo no histórico.

Uma regra automática ajuda a impedir esse erro.

34. **detect-private-key**

Outro hook apresentado é:

```
detect-private-key
```

Ele procura indícios de chaves privadas sendo adicionadas ao repositório.

Isso é especialmente importante porque chaves privadas podem permitir acesso indevido a recursos.

É um exemplo de como hooks podem funcionar como uma camada preventiva de segurança.

35. **check-merge-conflict**

Esse hook verifica sinais de conflitos de merge que tenham sido deixados no código.

Durante um conflito, podemos encontrar marcadores como:

```
<<<<<<<
=====
>>>>>>>
```

Se esses marcadores forem enviados acidentalmente para o repositório, o projeto pode ficar quebrado.

Uma verificação automática pode detectar esse tipo de problema antes do commit.

36. O que essa parte da aula realmente quer ensinar?

Não é apenas:

| "Aprenda três hooks."

O conceito mais importante é:

| Git pode ser integrado a ferramentas que automatizam políticas de qualidade, segurança e organização do projeto.

Podemos ter:

```
Código
↓
Qualidade
↓
Segurança
↓
Padronização
↓
Git
```

Tudo integrado ao fluxo de desenvolvimento.

37. O problema da dependência humana

Existe uma ideia recorrente nesta aula:

Regra manual

é mais frágil que:

Regra automatizada

Por exemplo:

Manual:

| "Lembre de verificar se não colocou uma chave."

Automatizado:

```
git commit
  ↓
detect-secrets
  ↓
verificação
```

Manual:

| "Lembre de escrever a mensagem no padrão."

Automatizado:

```
git commit
  ↓
commit-msg
  ↓
Commitizen
```

Manual:

| "Não coloque arquivos muito grandes."

Automatizado:

```
git commit
  ↓
check-added-large-files
```

38. Mas automação também possui limitações

É importante não cair no extremo de pensar:

| "Se temos pre-commit, o projeto está seguro."

Não.

Ferramentas automatizadas também precisam ser:

- configuradas;

- atualizadas;
- mantidas;
- executadas;
- complementadas por outras verificações.

Além disso, nem todo problema pode ser detectado automaticamente.

Por exemplo:

Código compila

não significa:

Código está bem projetado.

E:

Commit segue padrão

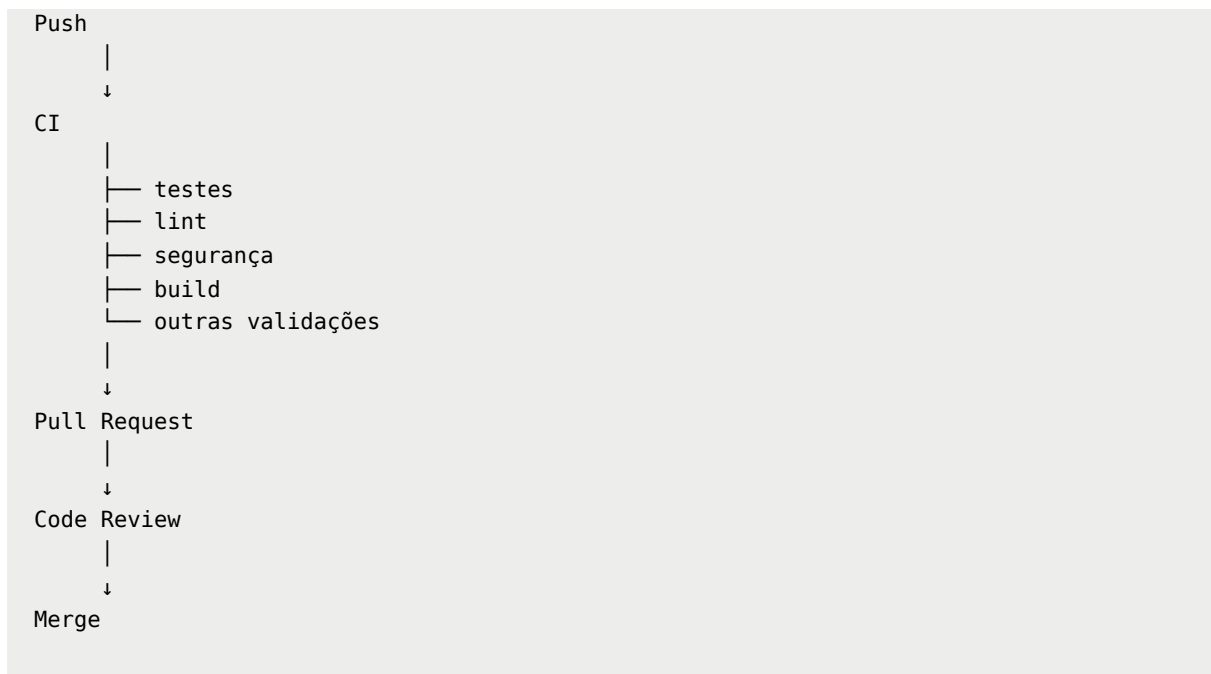
não significa:

Alteração é boa.

39. Automação local + automação coletiva

Uma arquitetura mais madura seria:

```
DESENVOLVEDOR
|
↓
Pre-commit
|
↓
Commit
|
↓
```



Cada etapa possui uma função.

40. O que o pre-commit resolve?

Principalmente:

| Detectar problemas cedo, no ambiente do desenvolvedor.

Ele pode impedir:

- formatação inadequada;
 - espaços desnecessários;
 - possíveis segredos;
 - arquivos grandes;
 - conflitos;
 - mensagens fora do padrão.
-

41. O que a CI resolve?

Principalmente:

| Garantir que as regras sejam verificadas de maneira coletiva e automatizada.

Mesmo que alguém:

- não tenha instalado o hook;
- tenha configurado errado;
- tenha esquecido de executar alguma ferramenta;

a CI pode executar novamente as verificações.

42. O que o Code Review resolve?

O Code Review permite que outras pessoas avaliem aspectos que ferramentas automáticas podem não compreender.

Por exemplo:

```
Ferramenta:  
"código está formatado."  
  
Revisor:  
"essa arquitetura realmente faz sentido?"
```

Outro exemplo:

```
Ferramenta:  
"testes passaram."  
  
Revisor:  
"o teste cobre o comportamento correto?"
```

Por isso automação e revisão humana se complementam.

43. Resposta conceitual para o exercício 2

A questão apresenta uma equipe que utiliza `pre-commit`, mas alguns desenvolvedores não executam os hooks.

A resposta deve mostrar que:

O pre-commit local é uma camada de prevenção, mas não deve ser a única garantia do projeto.

A equipe pode adotar:

1. Integração contínua

Executar automaticamente as mesmas validações no pipeline de CI.

Por exemplo:

```
Push
↓
CI
↓
lint
↓
testes
↓
segurança
↓
verificação de commits
```

2. Regras no repositório remoto

Configurar o projeto para exigir que determinadas verificações passem antes do merge.

3. Branch protection

Impedir que alterações sejam integradas diretamente sem passar pelas verificações exigidas.

4. Code Review

Exigir revisão antes da integração.

5. Padronização do ambiente

Documentar e automatizar o setup para que novos desenvolvedores instalem as ferramentas corretamente.

44. Uma resposta completa para a lógica do exercício

A ideia pode ser resumida assim:

```
Pre-commit local
→ prevenção

CI
→ validação automática

Servidor remoto
→ aplicação das regras

Code Review
→ avaliação humana

Documentação/setup
→ padronização
```

Assim, o projeto não depende exclusivamente de uma única ferramenta ou da disciplina individual.

45. O exercício 1: outros hooks úteis

O objetivo desse exercício não é simplesmente pesquisar nomes.

O professor quer que você entenda:

"Que outros problemas reais de desenvolvimento podem ser evitados automaticamente?"

Alguns tipos de hooks podem verificar:

- arquivos grandes;
- conflitos;
- formatação;
- lint;
- segredos;
- padrões de código;
- arquivos inválidos;
- sintaxe;
- convenções.

O importante em uma apresentação seria explicar:

```
Nome do hook
↓
O que ele verifica
↓
Qual problema evita
↓
Exemplo real
```

Por exemplo:

```
Hook:
check-added-large-files

Problema:
arquivo muito grande sendo adicionado

Caso real:
alguém adiciona acidentalmente um arquivo
de 500 MB ao repositório

Resultado:
hook bloqueia o commit
```

Essa estrutura já demonstra que você entendeu a ferramenta.

46. O exercício 3

O último exercício é mais aberto.

O professor quer que os alunos levem experiências reais com Git para discussão.

Isso é importante porque Git possui muitos problemas que não são facilmente compreendidos apenas lendo comandos.

Por exemplo, uma equipe pode discutir:

- conflitos de merge;
- branch errada;
- commit na branch errada;
- arquivos adicionados por engano;
- mensagens de commit inconsistentes;

- problemas de autenticação;
- histórico confuso;
- arquivos grandes;
- alterações que deveriam ter sido ignoradas.

A ideia é transformar experiências reais em aprendizado coletivo.

47. Relação com o que já estudamos sobre Git

Essa aula fecha vários assuntos anteriores.

Podemos montar uma evolução:

```
Git
↓
Controle de versão
↓
Branches
↓
Commits
↓
Trabalho em equipe
↓
Hooks
↓
Automação
↓
CI
↓
Code Review
↓
Processo profissional
```

Ou seja, Git deixa de ser apenas:

| "uma ferramenta para salvar versões."

E passa a fazer parte de um **processo de engenharia de software**.

48. Git no mercado

Em projetos profissionais, o Git normalmente está conectado a várias ferramentas.

Um fluxo pode ser:

```
Desenvolvedor
  ↓
IDE
  ↓
Git
  ↓
Pre-commit
  ↓
Commit
  ↓
Push
  ↓
GitHub/GitLab/Bitbucket
  ↓
CI/CD
  ↓
Testes
  ↓
Análise de qualidade
  ↓
Code Review
  ↓
Merge
  ↓
Deploy
```

Cada ferramenta possui uma responsabilidade.

Isso é muito diferente de simplesmente:

```
editar código
→ git add
→ git commit
→ git push
```

49. Uma distinção importante: ferramenta x processo

Outro conceito que vale muito para a prova:

Ferramenta não substitui processo.

Por exemplo:

```
pre-commit
```

é uma ferramenta.

Mas uma equipe precisa decidir:

- quais regras serão utilizadas;
- quais verificações são obrigatórias;
- onde elas serão executadas;
- quem pode fazer merge;
- quais falhas bloqueiam a integração.

Portanto:

```
Ferramenta  
+  
Configuração  
+  
Processo  
+  
Cultura  
=  
Fluxo de desenvolvimento mais confiável
```

50. O que acontece se a equipe configurar tudo errado?

Imagine:

```
pre-commit
```

está instalado.

Mas ninguém verifica os resultados.

Ou:

CI

detecta problemas.

Mas os desenvolvedores simplesmente ignoram os resultados.

Ou:

Code Review

existe.

Mas ninguém realmente revisa o código.

Nesse cenário, possuir as ferramentas não garante qualidade.

Por isso:

Ferramentas precisam estar integradas a um processo de desenvolvimento coerente.

51. O grande conceito da aula: Shift Left

Existe uma ideia importante por trás de muitas dessas ferramentas:

Detectar problemas o mais cedo possível.

Imagine:

```
Problema no código
  ↓
Desenvolvedor
  ↓
Pre-commit
  ↓
Detectado imediatamente
```

É melhor do que:

```
Problema
  ↓
Commit
  ↓
```

```
Push
↓
CI
↓
Review
↓
Descoberto
```

E melhor ainda do que:

```
Problema
↓
Produção
↓
Usuário encontra
```

Quanto mais cedo detectamos um problema, mais cedo podemos corrigi-lo.

O pre-commit é uma forma prática de aplicar essa ideia.

52. Mapa mental da aula

```
FERRAMENTAS DE APOIO AO GIT
|
├── Git Hooks
│   └── ações automáticas em eventos do Git
|
├── Pre-commit
│   ├── gerencia hooks
│   ├── formatação
│   ├── whitespace
│   ├── segredos
│   ├── arquivos grandes
│   └── conflitos
|
├── Commitizen
│   └── padronização de mensagens
│       └── Conventional Commits
|
├── Problema do pre-commit
│   └── execução local
|
├── Solução
│   ├── CI
│   ├── servidor remoto
│   ├── branch protection
│   ├── code review
│   └── setup padronizado
```

53. Perguntas que podem cair em prova

O que é um Git Hook?

É um mecanismo que permite executar ações automaticamente em determinados eventos do Git.

O que é pre-commit?

É um framework para gerenciar hooks e executar verificações automaticamente durante o fluxo de desenvolvimento, especialmente antes de commits.

Qual é a principal vantagem do pre-commit?

Detectar problemas cedo e automatizar regras do projeto, reduzindo a dependência de verificações manuais.

Qual é uma limitação do pre-commit?

Ele funciona localmente e depende de estar corretamente instalado e ativo no ambiente de cada desenvolvedor.

Por que o pre-commit não deve ser a única camada de validação?

Porque um desenvolvedor pode não instalar, configurar incorretamente ou contornar os hooks locais. A CI e as regras do repositório fornecem uma validação coletiva e independente do ambiente local.

O que é Commitizen?

É uma ferramenta que auxilia na criação de mensagens de commit padronizadas, podendo seguir convenções como Conventional Commits.

Por que padronizar mensagens de commit?

Para tornar o histórico mais organizado, compreensível e adequado para automações, como geração de changelogs.

Para que serve detect-secrets?

Para identificar possíveis segredos ou credenciais no código antes que sejam adicionados ao repositório.

Para que serve check-added-large-files?

Para impedir ou alertar sobre arquivos grandes sendo adicionados ao repositório.

Para que serve check-merge-conflict?

Para detectar indicadores de conflitos de merge que tenham sido deixados nos arquivos.

O que é CI?

Integração Contínua é uma prática em que alterações submetidas ao projeto passam automaticamente por verificações como build, testes, análise de código e segurança.

54. A pergunta mais importante da aula

Se o professor perguntar:

"Por que utilizar pre-commit se podemos simplesmente verificar tudo na CI?"

A resposta é:

Porque o pre-commit permite detectar problemas **mais cedo**, diretamente no ambiente do desenvolvedor, antes que a alteração seja enviada ao repositório.

A CI, por outro lado, funciona como uma segunda camada de garantia coletiva.

Então o ideal é:

```
Pre-commit  
→ feedback rápido  
  
CI  
→ garantia coletiva
```

Eles não competem.

Eles se complementam.

55. Resumo final

A aula apresenta ferramentas que ajudam a transformar regras de desenvolvimento em verificações automatizadas.

O conceito de **Git Hooks** permite executar ações automaticamente em determinados momentos do fluxo do Git.

O **pre-commit** facilita o gerenciamento dessas verificações.

Ele pode ser utilizado para:

- corrigir problemas de whitespace;
- verificar finais de arquivos;
- detectar possíveis segredos;
- impedir arquivos grandes;
- detectar conflitos;
- executar outras verificações.

O **Commitizen** ajuda a padronizar mensagens de commit, frequentemente utilizando convenções como:

```
feat:  
fix:  
docs:
```

Essa padronização melhora a organização do histórico e pode facilitar automações.

Entretanto, existe uma limitação fundamental:

```
pre-commit = local
```

Por isso, uma equipe profissional deve combinar:

```
Pre-commit  
+  
CI  
+  
Regras do repositório  
+  
Code Review  
+  
Padronização do ambiente
```

A grande ideia é:

Automatizar as regras importantes, detectar problemas cedo e não depender exclusivamente da disciplina individual de cada desenvolvedor.

E talvez a frase mais importante para guardar seja:

```
Pre-commit previne localmente.  
CI valida coletivamente.  
Code Review avalia o que a automação não consegue compreender.
```

Esse conjunto transforma o Git de uma simples ferramenta de versionamento em parte de um processo profissional de desenvolvimento de software.